ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ст. преподаватель		Н. А. Соловьева
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

СТРУКТУРЫ ДАННЫХ: МАССИВ, СПИСОК, ХЕШ-ТАБЛИЦА, МНОЖЕСТВО, СТЕК

по курсу: АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4516	30.09.26	И. С. Миронов
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Цель работы. Научиться реализовывать базовые структуры данных: массив, список, хеш-таблицу, множество и стек.

Задание на выполнение лабораторной работы. По таблице вариантов для варианта №14 нужно выполнить задания 6 и 12. В задании 6 необходимо реализовать структуру данных «Множество» на основе такой структуры данных, как «Односвязный список». Обращение к элементам множества должно производиться с использованием дандерных (магических) методов Python. Также реализовать 2 варианта проверки вхождения элемента в структуру данных, метод intersect для пересечения множеств и метод union для их объединения. В задании 12 необходимо реализовать структуру данных «Двусвязный список», где обращение к её элементам производится с использованием дандерных (магических) методов Python. Также реализовать 2 варианта проверки вхождения элемента в структуру данных. Каждое задание должно содержать минимум 5 тестов, 2 бенчмарка и быть реализовано с использованием Type Hinting.

1. **Задание 6. Множество на основе односвязного списка**

Множество строится на основе односвязного списка. Каждый узел хранит значение value и ссылку на следующий узел next. Класс MySet хранит только ссылку на голову списка head.

Метод add проходит по списку и проверяет, есть ли уже такое значение. Если есть — ничего не делает. Если нет — доходит до последнего узла и привязывает новый. Это защищает множество от дубликатов.

Доступ к элементам сделан через магические методы __len__ (для len()), __contains__ (для in) и __repr__ (для print()). Первый вариант проверки вхождения — магический __contains__, который вызывается оператором in. Второй вариант — обычный метод contains, который внутри тоже использует in.

Метод intersect создаёт новое множество и добавляет туда элементы первого множества, которые есть во втором. Метод union добавляет в новое множество все элементы из обоих множеств, а add автоматически отсекает повторы. Тесты для MySet приведены в листинге 2, их 6, они проверяют все методы класса.

Листинг 1 – Классы Node, MySet и магические методы

```
from typing import Optional
import time


class Node:
    def __init__(self, value: int, next: Optional['Node'] =
None) -> None:
        self.value = value
```

```
        self.next = next

class MySet:
    def __init__(self, head: Optional['Node']= None) -> None:
        self.head = head

    def add(self, value: int) -> None: #добавляем элемент,
если его ещё нет
        if self.head is None:
            self.head = Node(value)

        current = self.head
        while current is not None:
            if current.value == value:
                return
            if current.next is None:
                current.next = Node(value)
                return
            current = current.next

    def __len__(self)-> int: #вывод длины
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count

    def __contains__ (self, item: int) -> bool: #проверяем
вхождение числа в список через магический метод
        current = self.head
        while current is not None:
            if current.value == item:
                return True
            else:
                current = current.next
        return False

    def contains(self, item: int) -> bool: #проверяем
вхождение числа в список обычным способом
        return item in self

    def __repr__(self) -> str: #вывод чисел
        spis = []
        current = self.head
        while current is not None:
            spis.append(str(current.value))
            current = current.next
        return '{' + ', '.join(spis) + '}'
```

```
    def intersect(self, second_set: 'MySet') -> 'MySet':
#нахрдим пересечения
        current = self.head
        result = MySet()
        while current is not None:
            if current.value in second_set:
                result.add(current.value)
            current = current.next
        return result

    def union(self, second_set: 'MySet') -> 'MySet': #находим
объединения
        current = self.head
        result = MySet()
        while current is not None:
            result.add(current.value)
            current = current.next

        current = second_set.head
        while current is not None:
            result.add(current.value)
            current = current.next
        return result
```

Листинг 2 — Тесты для MySet (файл second_lab_test.py)

```
from second_lab import MySet
from second_lab import Node
from second_lab import DoublyLinkedList

# ТЕСТЫ ДЛЯ ОДНОСВЯЗНОГО СПИСКА

def test_add():
    a = MySet(Node(1, Node(2, Node(3, Node(4, Node(5))))))
    b = MySet()
    a.add(6)
    b.add(9)

    assert 6 in a
    assert str(a) == "{1, 2, 3, 4, 5, 6}"

    assert 9 in b
    assert str(b) == "{9}"

def test_len():
    a = MySet(Node(1, Node(2, Node(3, Node(4, Node(5))))))

    assert len(a) == 5

def test_contains():
    a = MySet(Node(1, Node(2, Node(3, Node(4, Node(5))))))
```

```
    assert 4 in a
    assert 7 not in a

def test_repr():
    a = MySet(Node(1, Node(2, Node(3, Node(4, Node(5))))))

    assert str(a) == "{1, 2, 3, 4, 5}"

def test_intersect():
    a = MySet(Node(1, Node(2, Node(3, Node(4, Node(5))))))
    b = MySet(Node(3, Node(9, Node(8, Node(1, Node(7))))))
    result = a.intersect(b)

    assert len(result) == 2
    assert 1 in result
    assert 3 in result
    assert 5 not in result
    assert 8 not in result

def test_union():
    a = MySet(Node(1, Node(2, Node(3, Node(4, Node(5))))))
    b = MySet(Node(3, Node(9, Node(8, Node(1, Node(7))))))
    result = a.union(b)

    assert 1 in result
    assert 2 in result
    assert 3 in result
    assert 4 in result
    assert 5 in result
    assert 7 in result
    assert 8 in result
    assert 9 in result
    assert len(result) == 8
```

2. **Задание 12. Двусвязный список**

Двусвязный список отличается от односвязного тем, что у каждого узла две ссылки: next на следующий узел и prev на предыдущий. Это позволяет ходить по списку в обе стороны.

Класс DoublyLinkedList хранит две ссылки — на голову head и на хвост tail. Наличие ссылки на хвост позволяет добавлять элементы в конец за O(1), без обхода всего списка. Метод add создаёт новый узел, привязывает его prev к текущему хвосту, next старого хвоста — к новому узлу, и делает новый узел хвостом. Если список пуст, новый узел становится и головой, и хвостом.

Доступ к элементам сделан через __len__, __contains__ и __repr__. Два варианта проверки вхождения — те же, что и у множества: магический __contains__ и обычный метод contains.

Листинг 3 — Классы Node2, DoublyLinkedList и их методы

```
class Node2:
    def __init__(self, value: int, next: Optional['Node2'] =
None,
                 prev: Optional['Node2'] = None) -> None:
        self.value = value
        self.next = next
        self.prev = prev


class DoublyLinkedList:
    def __init__(self) -> None:
        self.head: Optional[Node2] = None
        self.tail: Optional[Node2] = None

    def add(self, value: int) -> None:  # добавляем в конец
        new_node = Node2(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            return

        new_node.prev = self.tail
        self.tail.next = new_node
        self.tail = new_node

    def __len__(self) -> int:  # вывод длины
        count = 0
        current = self.head
        while current is not None:
            count += 1
            current = current.next
        return count

    def __contains__(self, item: int) -> bool:  # проверка
вхождения (магический метод)
        current = self.head
        while current is not None:
            if current.value == item:
                return True
            current = current.next
        return False

    def contains(self, item: int) -> bool:  # проверка
вхождения (обычный метод)
```

```
        return item in self

    def __repr__(self) -> str:  # вывод чисел
        spis = []
        current = self.head
        while current is not None:
            spis.append(str(current.value))
            current = current.next
        return '{' + ', '.join(spis) + '}'
```

Тесты для DoublyLinkedList приведены в листинге 4, их 6, они проверяют все методы класса.

Листинг 4 - Тесты для DoublyLinkedList (файл second_lab_test.py)

```
# ТЕСТЫ ДЛЯ ДВУХСВЯЗНОГО СПИСКА

def test_doubly_add():
    d = DoublyLinkedList()
    d.add(1)
    d.add(2)
    d.add(3)

    assert 1 in d
    assert 2 in d
    assert 3 in d
    assert len(d) == 3


def test_doubly_len():
    d = DoublyLinkedList()
    d.add(1)
    d.add(2)
    d.add(3)
    d.add(4)
    d.add(5)

    assert len(d) == 5


def test_doubly_contains():
    d = DoublyLinkedList()
    d.add(1)
    d.add(2)
    d.add(3)

    assert 2 in d          # магический метод
    assert 99 not in d


def test_doubly_contains_method():
```

```
    d = DoublyLinkedList()
    d.add(1)
    d.add(2)
    d.add(3)

    assert d.contains(3) is True    # обычный метод
    assert d.contains(99) is False


def test_doubly_repr():
    d = DoublyLinkedList()
    d.add(1)
    d.add(2)
    d.add(3)

    assert str(d) == "{1, 2, 3}"

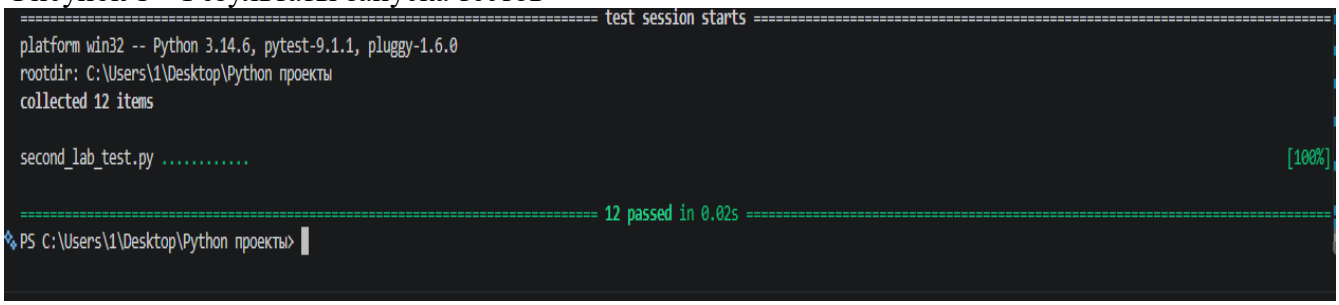

def test_doubly_empty():
    d = DoublyLinkedList()

    assert len(d) == 0
    assert str(d) == "{}"
    assert 5 not in d
```

3. Результаты тестирования

Тесты написаны на pytest. Все 12 тестов прошли успешно (Рис. 1).

Рисунок 1 – Результаты запуска тестов



4. Бенчмарки

Время замерялось функцией time.time(). Размер выборки — 10000 элементов. Бенчмарки 1 и 2 относятся к заданию 6, бенчмарки 3 и 4 — к заданию 12.

Листинг 5 — Бенчмарки (внутри if __name__ == '__main__' в second_lab.py)

```
if __name__ == '__main__':
    head = Node(1, Node(2, Node(3, Node(4, Node(5)))))
    second_set = Node(3, Node(9, Node(8, Node(1, Node(7)))))

    a1 = MySet(head)
    a2 = MySet(second_set)
```

```
print('==========================================================
============================')
    print('====================  Результаты работы
односвязного списка ======================')

print('==========================================================
============================')

    print(a1)
    print(a2)
    print(len(a1))
    print(5 in a1)
    print(99 in a1)
    print(a1.contains(5))
    print(a1.intersect(a2))
    print(a1.union(a2))

print('==========================================================
============================')
    print('====================  Результаты работы
двухсвязного списка ======================')

print('==========================================================
============================')


    d = DoublyLinkedList()
    d.add(1)
    d.add(2)
    d.add(3)
    print(d)
    print(len(d))
    print(2 in d)
    print(99 in d)
    print(d.contains(3))

    print('======================== Бэнчмарки для односвязного
списка ========================')

    start = time.time()

    a = MySet()
    for i in range(10000):
        a.add(i)

    end = time.time()
    print(f"Добавление 10000 элементов: {end - start:.4f}
секунд")
```

```
    a = MySet()
    for i in range(10000):
        a.add(i)

    start = time.time()

    result = 9999 in a

    end = time.time()
    print(f"Поиск последнего элемента: {end - start:.6f}
секунд")
    print(f"Найдено: {result}")

    print('======================== Бэнчмарки для двухсвязного
списка ========================')

    start = time.time()

    d = DoublyLinkedList()
    for i in range(10000):
        d.add(i)

    end = time.time()
    print(f"Двусвязный: добавление 10000 элементов: {end -
start:.4f} секунд")

    d = DoublyLinkedList()
    for i in range(10000):
        d.add(i)

    start = time.time()
    result = 9999 in d
    end = time.time()

    print(f"Двусвязный: поиск последнего элемента: {end -
start:.6f} секунд")
    print(f"Найдено: {result}")
```

Рисунок 2 - Результаты замеров

```
======================== Бэнчмарки для односвязного списка ========================
Добавление 10000 элементов: 2.3190 секунд
Поиск последнего элемента: 0.000327 секунд
Найдено: True
======================== Бэнчмарки для двухсвязного списка ========================
Двусвязный: добавление 10000 элементов: 0.0042 секунд
Двусвязный: поиск последнего элемента: 0.000414 секунд
Найдено: True
PS C:\Users\1\Desktop\Python проекты>
```

Выводы

В ходе работы я реализовал две структуры данных: множество на основе односвязного списка и двусвязный список. Обе структуры используют дандерные методы (__len__, __contains__, __repr__) и аннотации типов. Для обеих сделал по два варианта проверки вхождения. Написано 12 тестов на pytest, все проходят.

По бенчмаркам видно, что двусвязный список добавляет элементы значительно быстрее, чем множество на односвязном списке. Это связано с тем, что DoublyLinkedList хранит ссылку на хвост (tail), поэтому add работает за O(1) — просто привязывает новый узел к хвосту. В MySet хвоста нет, поэтому add каждый раз обходит весь список от головы до конца, что даёт O(n).

Поиск элемента (__contains__) в обеих структурах имеет одинаковую сложность O(n) — приходится идти от головы до совпадения или до конца списка. Время поиска примерно одинаковое, потому что логика обхода одинаковая.

Для больших данных односвязное множество показывает себя плохо из-за квадратичной сложности заполнения. Двусвязный список с хвостом в этом плане эффективнее. Если бы в множестве тоже хранился хвост, его add работал бы за O(1), но при этом пришлось бы дополнительно проверять дубликаты — а это всё равно обход всего списка, то есть O(n).